Lab 1 Report: Task 10 and Task 11

William Lebrato, aile20

1 Implementation

This section describes the parallelization strategies used for Gaussian elimination (Task 10) and Quicksort (Task 11). Both algorithms rely on the same underlying design: a persistent **thread pool** paired with a **FIFO task queue**, ensuring that threads remain active for the entire runtime and avoid repeated creation and destruction.

1.1 Gaussian Elimination

The sequential version follows Algorithm 8.4 (Grama et al.), normalizing pivot rows and eliminating remaining rows for each iteration. The parallel implementation keeps the pivot-row division step sequential but distributes elimination among worker threads.

Threading Model and Task Structure

Each elimination phase enqueues one task per worker thread, encoding the pivot index k and thread ID. Workers pull tasks from the queue, perform row elimination, and signal completion. Since the thread pool persists throughout the entire algorithm, only pivot-level synchronization is required.

Cyclic Row Distribution

Work is divided according to:

$$i = k + 1 + \texttt{thread\_id}, \qquad i \mathrel{+}= \texttt{NUM\_THREADS},$$

ensuring a balanced distribution of rows across workers.

1.2 Quicksort

The parallel Quicksort implementation uses the same thread-pool infrastructure but applies it to recursive divide–conquer tasks.

Parallel Array Initialization

The array of $64 \cdot 2^{20}$ integers is initialized in parallel by assigning each thread a contiguous segment. Random numbers are generated using `rand_r()` to guarantee thread safety.

Partitioning and Task Creation

After partitioning, the right subproblem is handled by the current worker, while the left subproblem becomes a task when the recursion depth is below `MAX_DEPTH`. Beyond the threshold, both sides execute sequentially to avoid excessive task generation.

Thread Pool Operation

Workers repeatedly dequeue tasks, execute Quicksort on the assigned segment, update task counters, and signal completion. The main thread enqueues the initial task and waits for all work to finish.

2 Results

The measurements where collected by executing each program 5 times, calculating the average.

2.1 Gaussian Elimination

Correctness was verified using the provided validation script. All tested matrix sizes up to 2048×2048 produced identical results.

Table 1: Gaussian Elimination Execution Times

Implementation	**Average Time (s)**	**Speedup**
Sequential	2.897	1.00
Parallel	1.883	1.54

2.2 Quicksort

Table 2: Quicksort Execution Times and Speedup

Configuration	**Average Time (s)**	**Speedup**
Sequential	8.166	1.00
Parallel, 4 threads	6.234	1.31×
Parallel, 8 threads	4.236	1.93×
Parallel, 16 threads (Depth 4)	4.356	1.87×
Parallel, 16 threads (Depth 6)	3.496	2.34×
Parallel, 16 threads (Depth 8)	2.957	2.76×
Parallel, 16 threads (Depth 10)	2.227	3.67×
Parallel, 16 threads (Depth 12)	2.075	3.94×
Parallel, 16 threads (Depth 14)	1.859	4.39×

3 Analysis

3.1 Gaussian Elimination

Runtime is dominated by the elimination phase, which parallelizes efficiently. The persistent thread pool removes thread creation overhead, leaving only synchronization and task dispatch costs. Cyclic row distribution ensures balanced workloads across workers, resulting in consistent performance that meets the requirement of achieving more than 1.5× speedup on 16 cores.

3.2 Quicksort

Results show a direct correlation between recursion depth and efficiency. Shallow depths (e.g., 4) generate fewer tasks, causing load imbalances where threads sit idle after uneven pivots. Increasing `MAX_DEPTH` creates more fine-grained tasks, keeping the shared queue populated. This ensures constant utilization, as workers can immediately dequeue new subproblems upon completion. This improved load balancing allowed the implementation to surpass the target with a 4.39× speedup at depth 14.